Diagramme entité-relation de la base de données

Date : 2026-10-03 **Référence :** §II.C “Relations entre les tables” du manuel de certification. **Source de vérité :** `crates/pos-db/src/schema.rs`, `schema_hubrise.rs`, `schema_catalog.rs`, `schema_eft.rs` et `schema_badges.rs`, assemblés par `crates/pos-db/src/migrate.rs` (le SQL définit le schéma exact ; ce document reflète la révision actuelle : migrations 1 à 11, puis 20 à 28, puis 70).

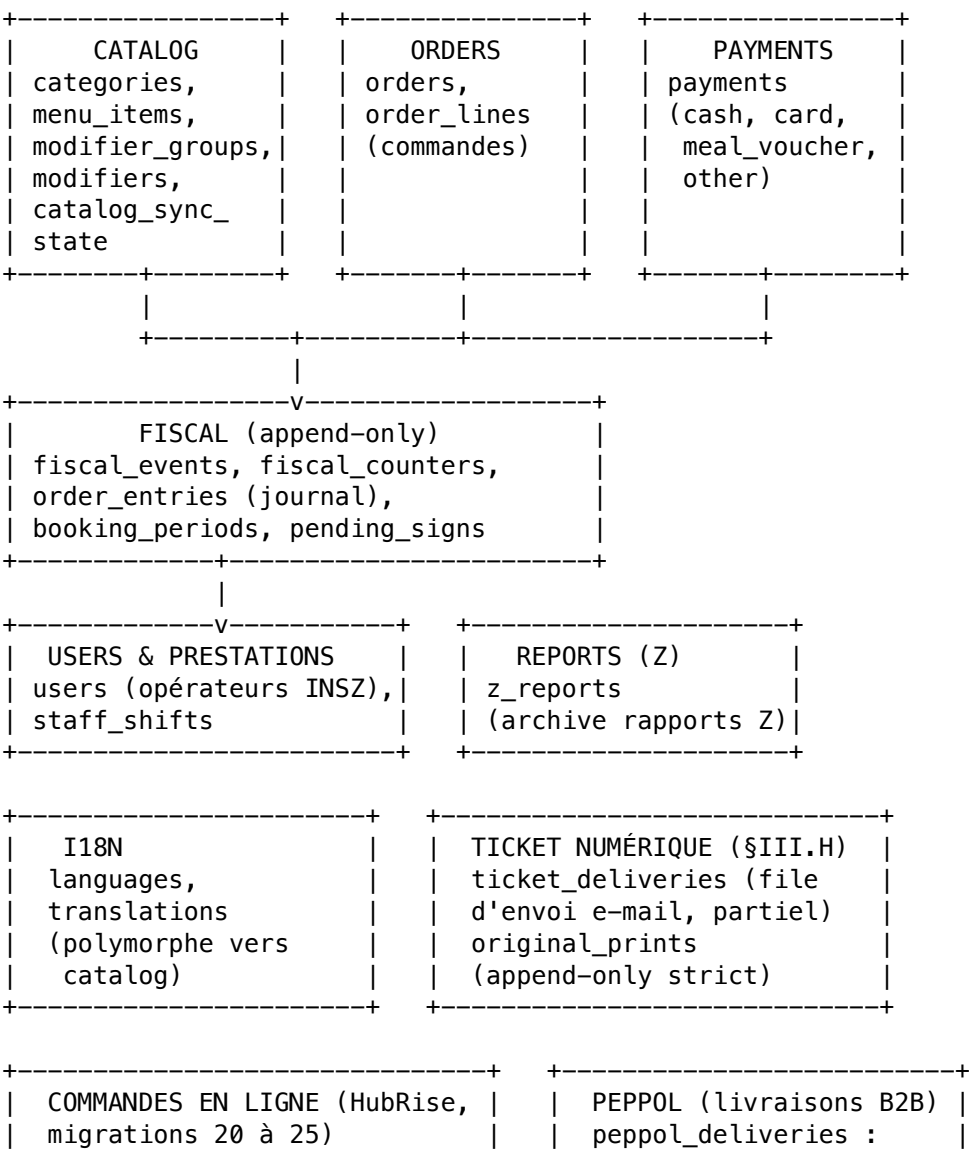
Ce document décrit visuellement les relations entre les tables de la base SQLite `pos.db`. Pour l’inventaire détaillé champ-par-champ, *voir : SCHEMA_BD_FR.md*. Pour la sécurité et les contraintes, *voir : SECURITE_BD_FR.md*.

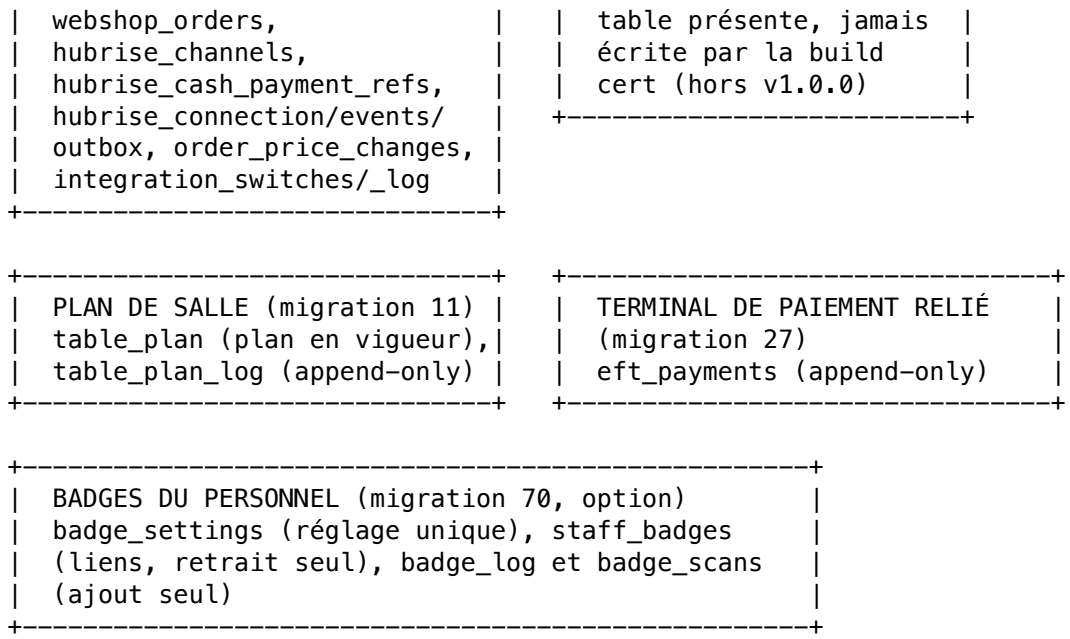
Une note sur les clés : les colonnes `user_insz` (dans `orders`, `fiscal_events`, `pending_signs`, `staff_shifts`) contiennent l’INSZ de l’opérateur mais ne portent **aucune clé étrangère** vers `users` : un événement fiscal garde son INSZ même si l’utilisateur est désactivé ou retiré. Le lien est applicatif.

---

1. Vue d’ensemble par domaines

La base est organisée en 13 domaines logiques :



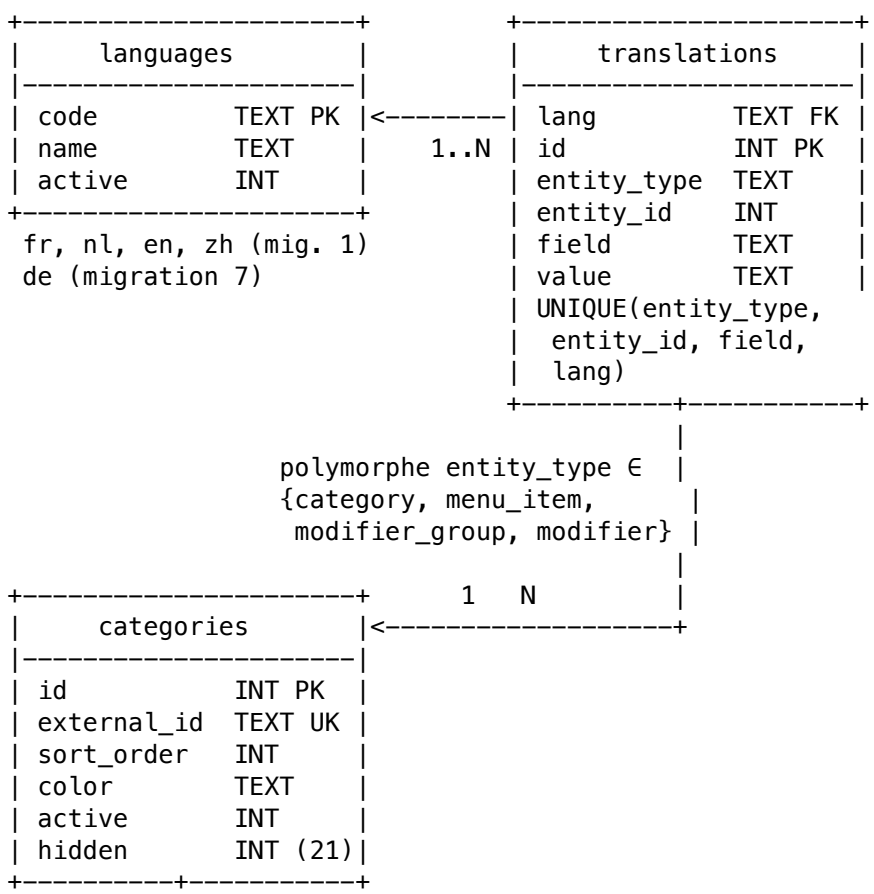


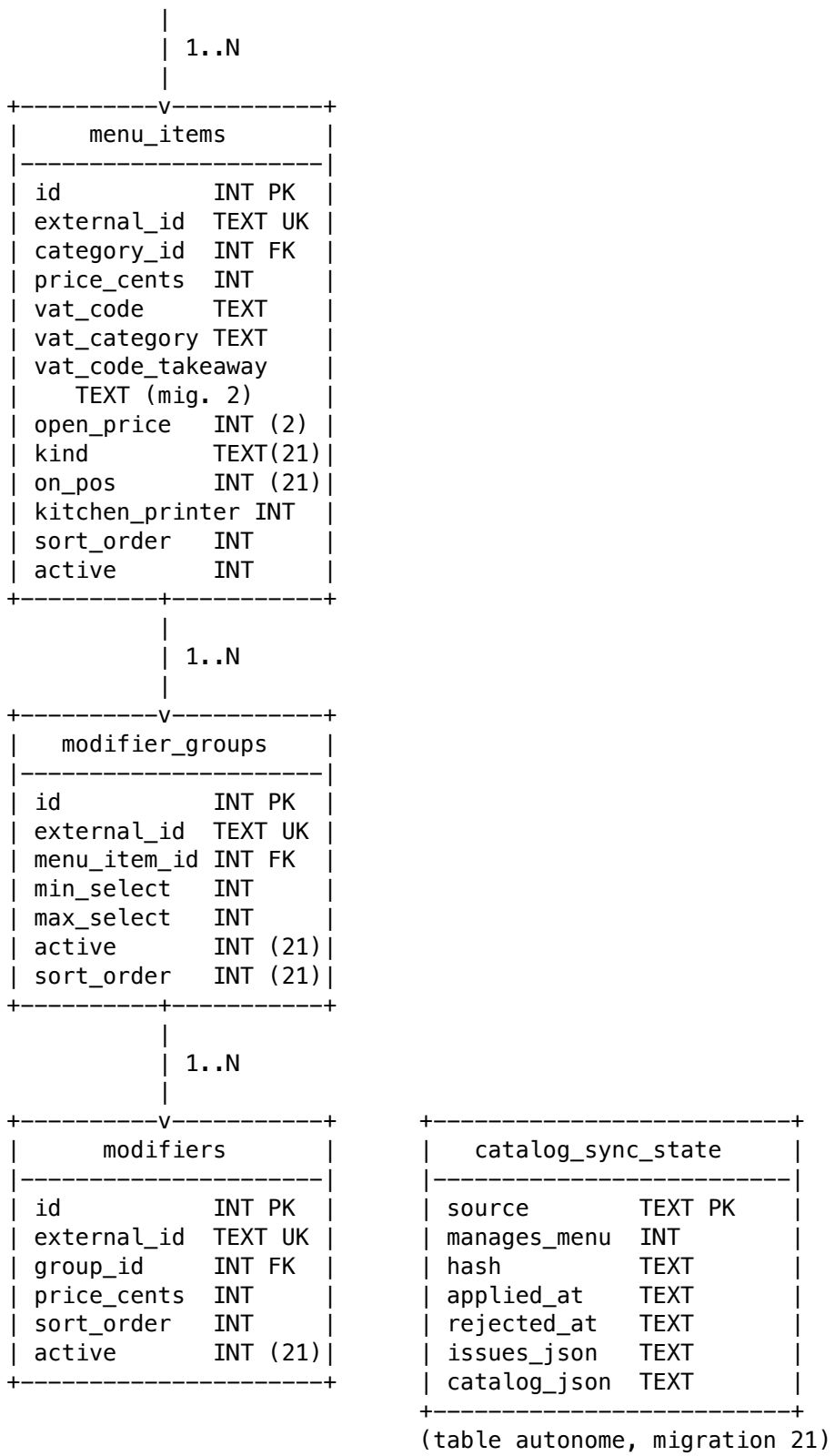
`_migrations_applied` (suivi des migrations) est hors domaine.

---

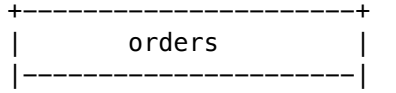
2. Diagrammes ER détaillés

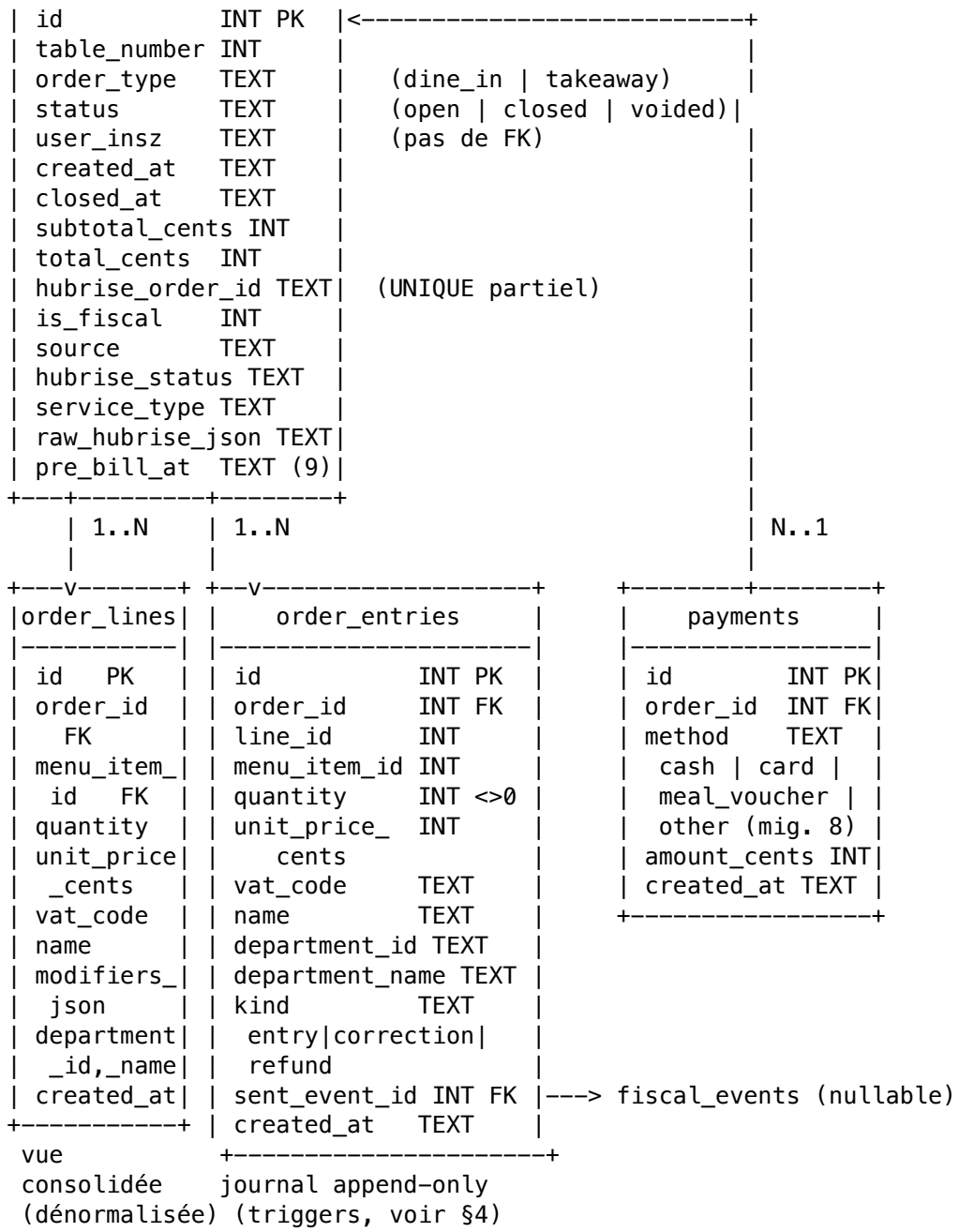
2.1 Catalogue et traductions



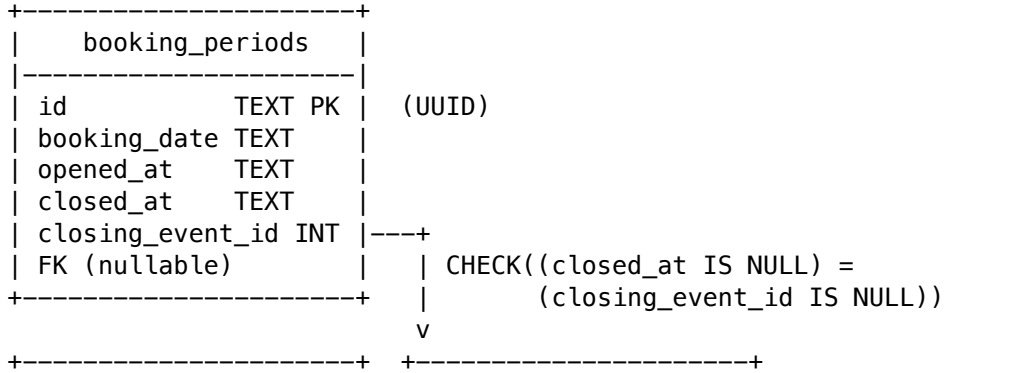


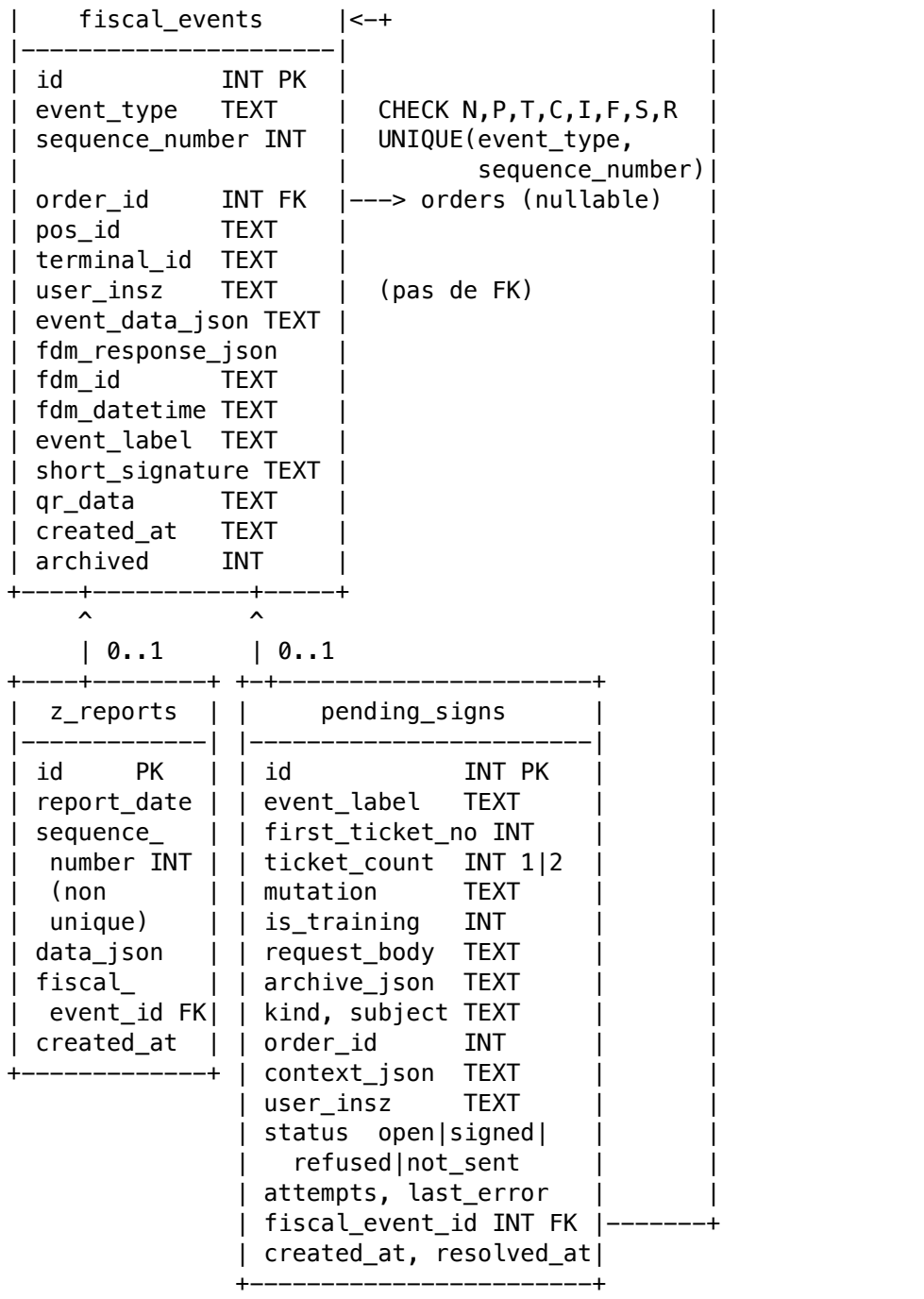
2.2 Commandes, lignes, paiements, journal





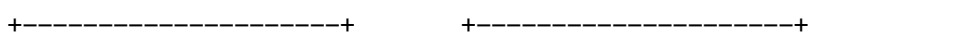
2.3 Fiscal, périodes, signatures en attente, rapports Z

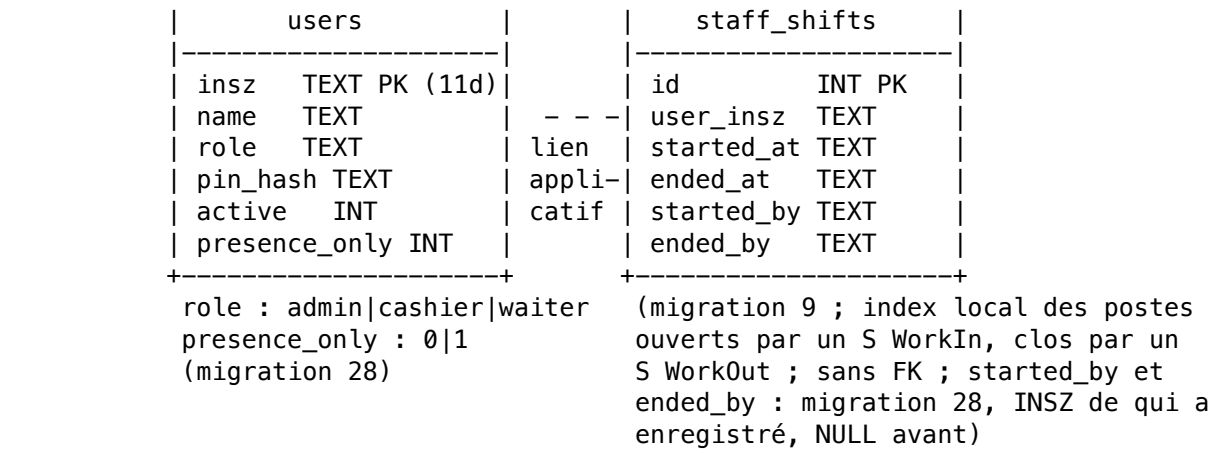




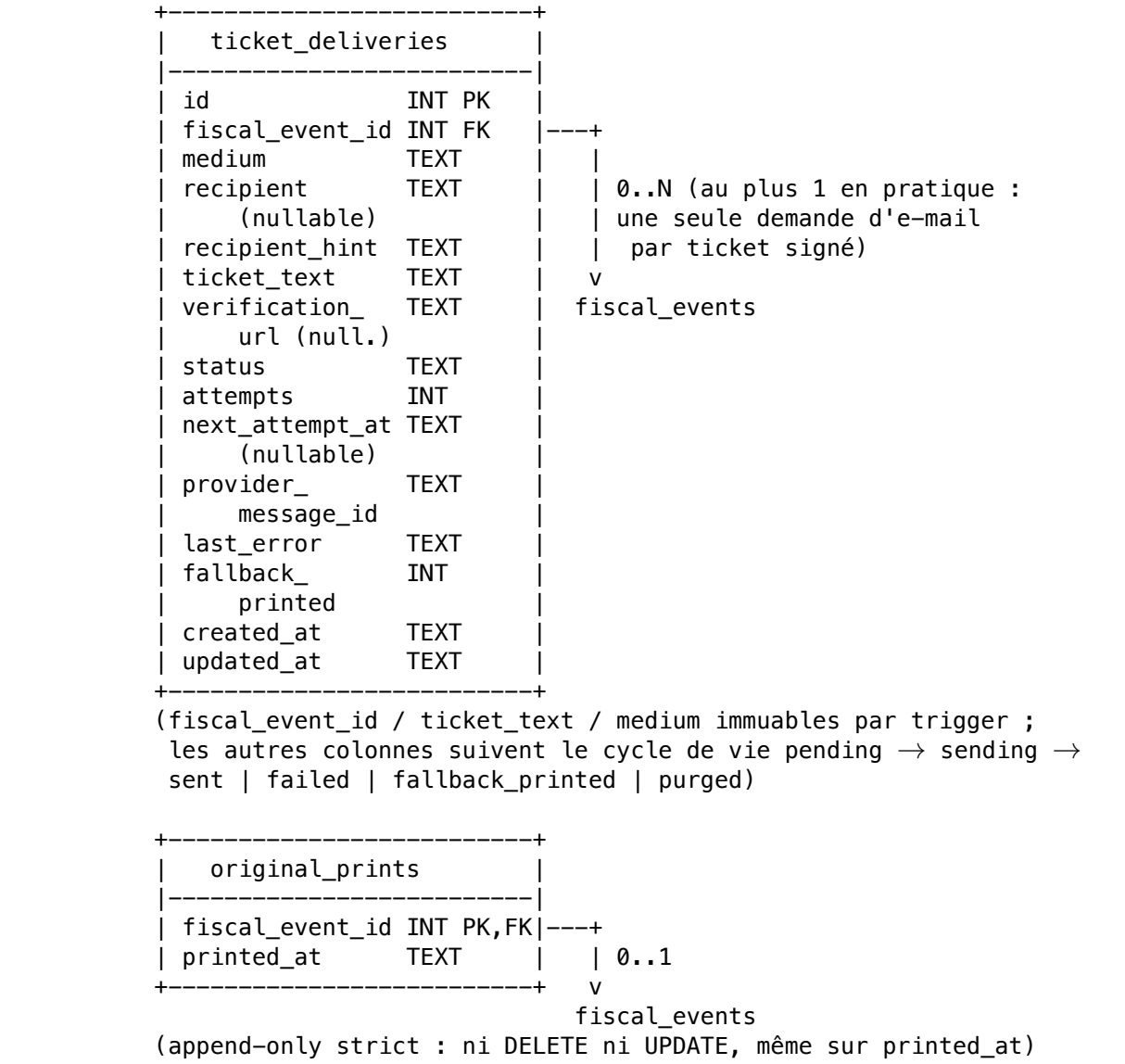
(table autonome, mise à jour atomique dans la même transaction
SQLite qui ajoute à fiscal_events, sous le verrou de signature)

2.4 Utilisateurs et prestations

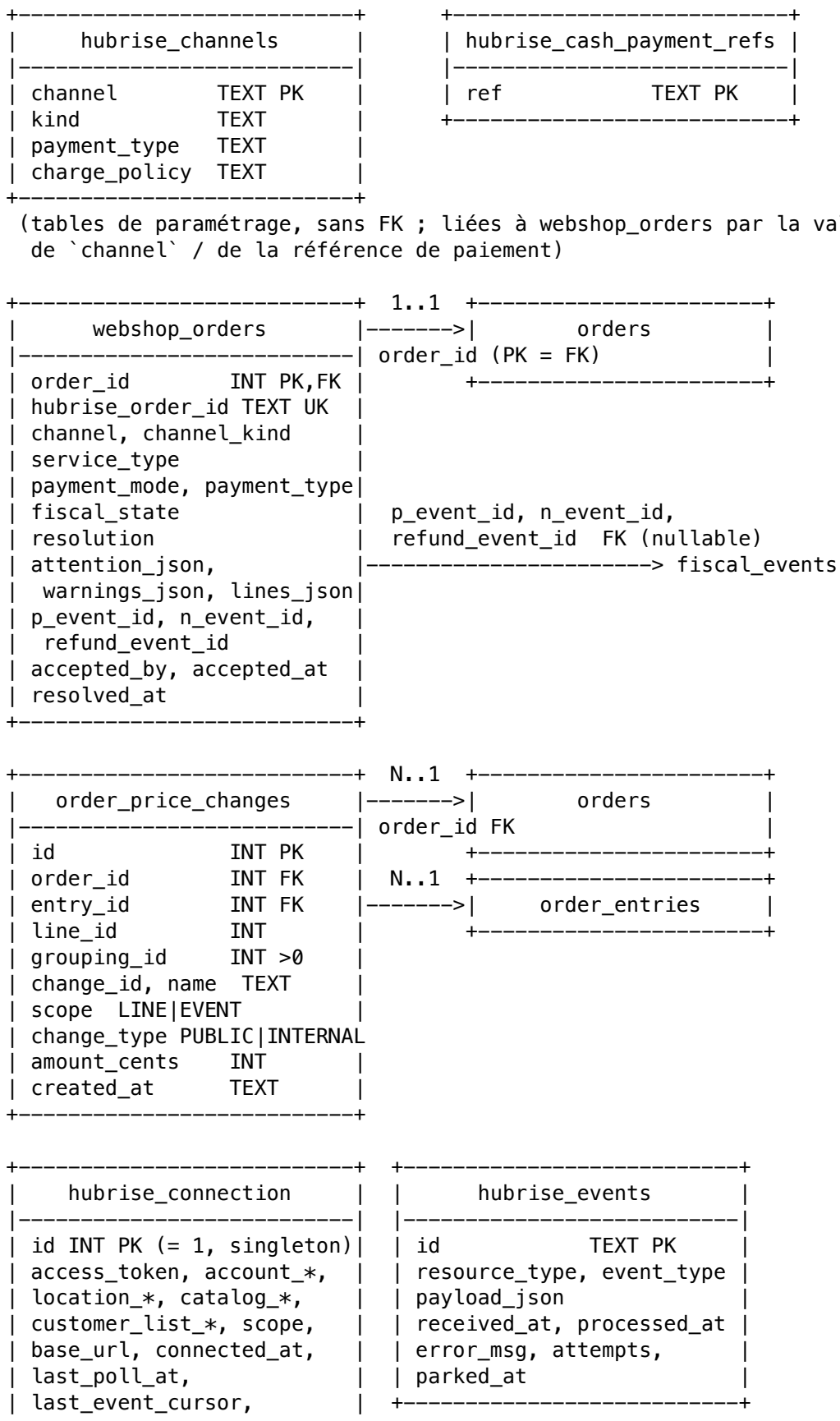


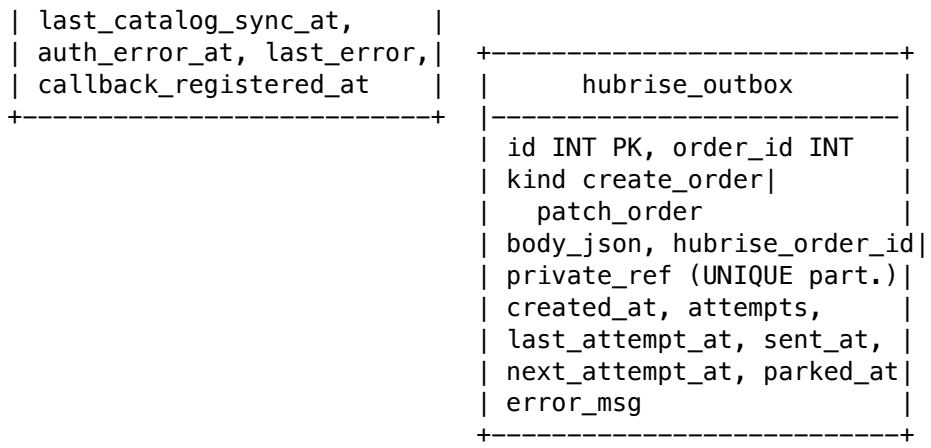


2.5 Ticket numérique et impressions d'original

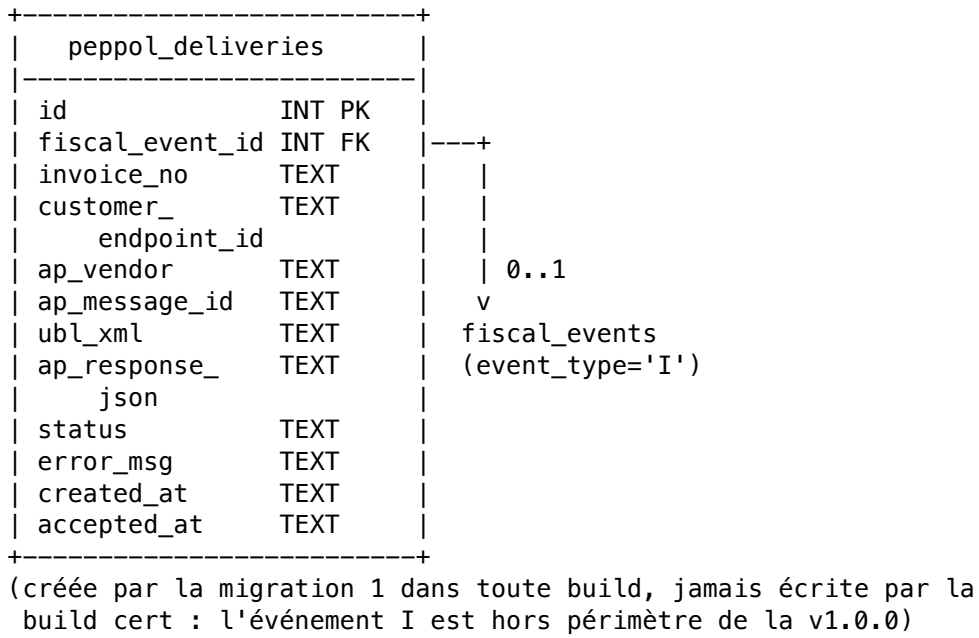


2.6 Commandes en ligne (HubRise, migrations 20 à 25)

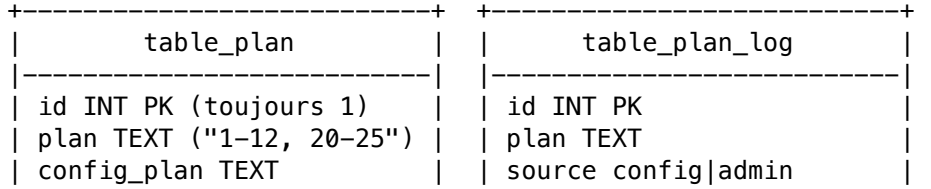


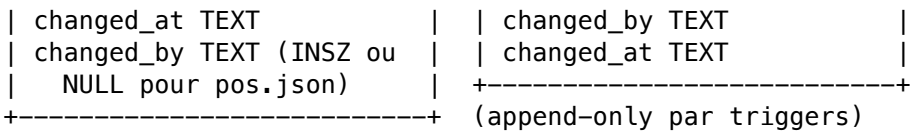


2.7 Peppol (hors binaire certifié v1.0.0)



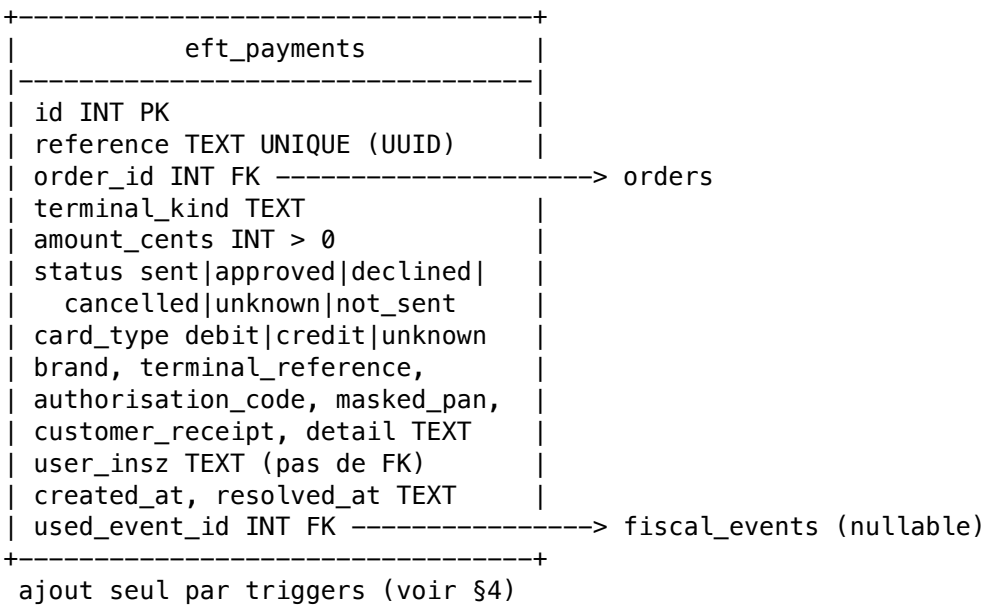
2.8 Plan de salle (migration 11)





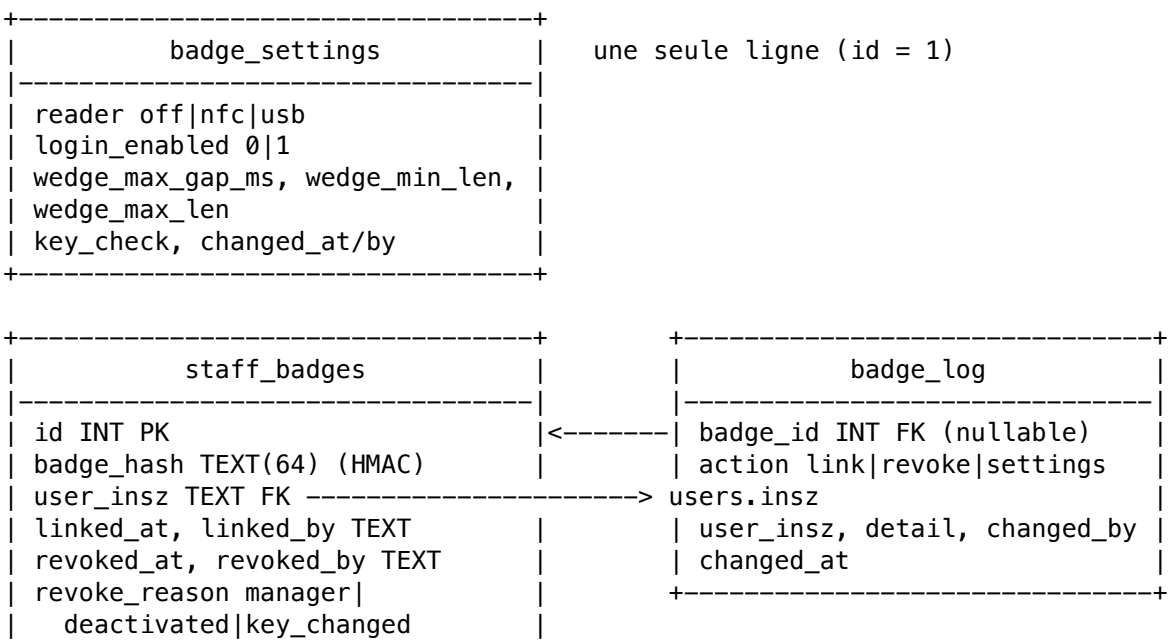
Aucune clé étrangère : le plan ne porte que des numéros de table, et `changed_by` est l'INSZ du Gérant (lien applicatif). Les commandes gardent leur numéro de table (`orders.table_number`) ; le plan ne détermine que les tables que l'écran des tables affiche, plus toute table hors plan qui a une commande ouverte.

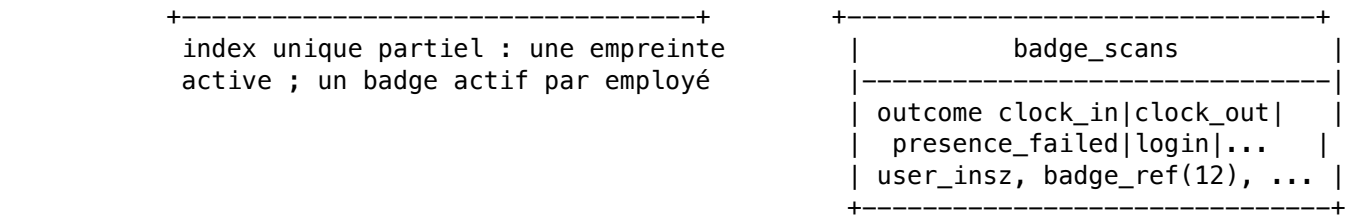
2.9 Terminal de paiement relié (migration 27)



Une ligne par paiement demandé au terminal, écrite avant l'appel. Elle référence sa commande et, une fois la vente signée, l'événement N qui l'a réglée (`used_event_id`, une seule fois). Un paiement `approved` impose un `card_type` (`CHECK`).

2.10 Badges du personnel (migration 70)





`staff_badges.user_insz` est, contrairement aux colonnes `user_insz` des tables fiscales, une clé étrangère vers `users` : un badge ne désigne qu'un employé connu. Les journaux `badge_log` et `badge_scans` gardent l'INSZ sans clé étrangère. Le numéro du badge n'apparaît dans aucune colonne (`SECURITE_BD_FR.md` §2.5).

---

3. Relations principales (verbalisées)

De → Vers	Cardinalité	Sémantique
`categories` → `menu_items`	1 → N	Une catégorie contient N articles de menu
`menu_items` → `modifier_groups`	1 → N	Un article peut avoir N groupes de modificateurs (sauce, cuisson, …)
`modifier_groups` → `modifiers`	1 → N	Chaque groupe contient N options (Soja, Wasabi, …)
`orders` → `order_lines`	1 → N	Une commande contient N lignes
`order_lines` → `menu_items`	N → 1	Référence vers l'article (nom + prix copiés, figés)
`orders` → `order_entries`	1 → N	Le journal chronologique de la commande (ajouts, corrections, remboursements)
`order_entries` → `fiscal_events`	N → 0..1	`sent_event_id` : l'événement P ou N qui a transmis l'entrée
`orders` → `payments`	1 → N	Une commande peut être réglée par N paiements (split, titres-repas)
`orders` → `eft_payments`	1 → 0..N	Les paiements demandés au terminal relié pour cette commande (aucun sans terminal relié)
`eft_payments` → `fiscal_events`	N → 0..1	`used_event_id` : l'événement N qui a réglé le paiement accepté
`orders` → `fiscal_events`	1 → 0..N	Une commande peut générer 0 (open) à plusieurs events fiscaux (P → N, ou N + C copie, etc.)
`fiscal_events` → `orders`	N → 0..1	Un event fiscal référence 0 (events système comme S, F-WAR) ou 1 commande
`users` → `orders` / `fiscal_events` / `pending_signs` / `staff_shifts`	1 → N	Par la valeur de `user_insz` ; **aucune clé étrangère** (lien applicatif)
`booking_periods` → `fiscal_events`	0..1 → 1	`closing_event_id` : l'événement R qui clôt la période ; un événement ne peut être archivé dans une période close

De → Vers	Cardinalité	Sémantique
`pending_signs` → `fiscal_events`	0..1 → 1	`fiscal_event_id` : l'événement archivé (premier événement si `ticket_count = 2`)
`z_reports` → `fiscal_events`	0..1 → 1	Chaque Z archive un event R correspondant. `sequence_number` n'est pas unique (la contrainte d'unicité est portée par `fiscal_events`)
`peppol_deliveries` → `fiscal_events`	0..1 → 1	Liée à un event I ; table jamais écrite par la build cert
`ticket_deliveries` → `fiscal_events`	0..N → 1	Une ligne par ticket dont le support demandé inclut l'e-mail (`DIGITAL`/`PAPER_DIGITAL`) ; ajoutée dans la même transaction que l'event N/T/C correspondant
`original_prints` → `fiscal_events`	0..1 → 1	Au plus une ligne par événement dont l'original a été imprimé (contrainte de clé primaire = garde applicative contre une seconde impression)
`webshop_orders` → `orders`	1 → 1	Une ligne par commande en ligne, à côté de sa ligne `orders`
`webshop_orders` → `fiscal_events`	N → 0..1	`p_event_id`, `n_event_id`, `refund_event_id`
`order_price_changes` → `orders`, `order_entries`	N → 1	Remises d'une commande en ligne, par ligne du journal ; `line_id` désigne la ligne d'`order_lines`
`translations` → (polymorphe)	N → 1	Traduction d'un champ d'une entité (category / menu_item / modifier_group / modifier) dans une langue (FR / NL / DE / EN / ZH)
`translations` → `languages`	N → 1	Référence le code langue
`fiscal_counters` (autonome)	-	Pas de FK ; clé `event_label` ∈ {N, P, T, C, I, F, S, R}

---

4. Append-only et intégrité

Les tables ci-dessous sont protégées **au niveau de la base** par des triggers SQLite (`RAISE(ABORT, ...)`), en plus de la discipline applicative :

Table	Triggers	Effet
`fiscal_events`	`fiscal_events_no_delete`, `fiscal_events_no_rewrite`	Ni DELETE, ni UPDATE
`fiscal_events`	`fiscal_events_period_open` (BEFORE INSERT)	Refuse un événement dont le `bookingPeriodId` désigne une période close
`z_reports`	`z_reports_no_delete`, `z_reports_no_rewrite`	Ni DELETE, ni UPDATE

Table	Triggers	Effet
order_entries	order_entries_no_delete, order_entries_no_rewrite	Pas de DELETE ; UPDATE interdit sauf sent_event_id
booking_periods	booking_periods_no_reopen, booking_periods_no_delete	Une période close ne change plus ; pas de DELETE
ticket_deliveries	ticket_deliveries_no_delete, ticket_deliveries_no_rewrite	Pas de DELETE ; fiscal_event_id, ticket_text, medium immuables (les autres colonnes suivent le cycle de vie de l'envoi : append-only **partiel**)
original_prints	original_prints_no_delete, original_prints_no_update	Ni DELETE, ni UPDATE, aucune exception
pending_signs	pending_signs_request_immutable, pending_signs_context_on_resolution, pending_signs_no_reopen, pending_signs_no_delete	Ce qui a été envoyé est immuable ; une ligne résolue ne change plus ; pas de DELETE
order_price_changes	order_price_changes_no_delete, order_price_changes_no_update	Ni DELETE, ni UPDATE
integration_switch_log	integration_switch_log_no_update, integration_switch_log_no_delete	Ni DELETE, ni UPDATE
table_plan_log	table_plan_log_no_update, table_plan_log_no_delete	Ni DELETE, ni UPDATE
eft_payments	eft_payments_no_delete, eft_payments_starts_sent, eft_payments_request_fixed, eft_payments_status_moves, eft_payments_answer_final, eft_payments_used_once	Pas de DELETE ; une ligne naît sent et non utilisée ; la demande (commande, montant, référence, opérateur) est immuable ; seuls sent et unknown changent de statut ; une réponse définitive ne change plus (hors used_event_id, posé une fois pour un paiement approved)
staff_badges	staff_badges_no_delete, staff_badges_revoke_only	Pas de DELETE ; la seule modification possible est un retrait, une fois (l'empreinte, l'INSZ et les données de lien sont immuables)
badge_log	badge_log_no_update, badge_log_no_delete	Ni DELETE, ni UPDATE
badge_scans	badge_scans_no_update, badge_scans_no_delete	Ni DELETE, ni UPDATE

Soit 35 triggers.

Pour les autres tables (catalogue, commandes en cours, paiements, utilisateurs, staff_shifts, tables HubRise hors journal) les opérations UPDATE/DELETE sont autorisées via la couche commandes. Les orders sont marquées status='closed' ou status='voided' plutôt que supprimées. Le catalogue synchronisé met à jour ses lignes en place ; l'historique fiscal ne dépend pas de ces lignes (nom et prix sont figés dans order_lines / order_entries).

La contrainte UNIQUE(event_type, sequence_number) de fiscal_events empêche tout doublon de compteur fiscal au niveau base, par type d'événement. La table fiscal_counters est mise à jour atomiquement dans la même transaction SQLite que l'insert dans fiscal_events, sous le verrou de signature unique (cf. *STRATEGIE_COMPTEURS_FR.md* §5). Les requêtes de signature en attente de réponse sont conservées

dans `pending_signs` (cf. *SCHEMA_BD_FR.md* §3.9).

---

5. Indexes

Index explicites (hors index implicites des clés primaires et des contraintes UNIQUE) :

Table	Index	Colonnes	Raison
`translations`	`idx_translations_lookup`	`entity_type, entity_id, lang`	Lookup rapide pour le rendu UI multilingue
`orders`	`idx_orders_hubrise_order_id` (unique partiel, `WHERE hubrise_order_id IS NOT NULL`)	`hubrise_order_id`	Ingestion idempotente des commandes en ligne
`order_entries`	`idx_order_entries_order`	`order_id, id`	Relire le journal d'une commande dans l'ordre
`booking_periods`	`idx_booking_periods_one_open` (unique partiel, `WHERE closed_at IS NULL`)	`(closed_at IS NULL)`	Au plus une période ouverte
`ticket_deliveries`	`idx_ticket_deliveries_due`	`status, next_attempt_at`	Sélection rapide des envois dus par le worker de réessai (§III.H)
`staff_shifts`	`idx_staff_shifts_open`	`user_insz, ended_at`	Retrouver le poste ouvert d'un employé
`pending_signs`	`idx_pending_signs_one_open` (unique partiel, `WHERE status = 'open'`)	`(status)`	Au plus une requête de signature ouverte
`pending_signs`	`idx_pending_signs_order`	`order_id, status`	Retrouver la requête en attente d'une commande
`peppol_deliveries`	`idx_peppol_deliveries_invoice_no`		Recherche par numéro de facture (table inutilisée en v1.0.0)
`hubrise_events`	`idx_hubrise_events_pending` (partiel, `WHERE processed_at IS NULL`)	`received_at`	Événements HubRise restant à traiter
`hubrise_outbox`	`idx_hubrise_outbox_pending` (partiel, `WHERE sent_at IS NULL`)	`created_at`	Statuts restant à envoyer
`hubrise_outbox`	`idx_hubrise_outbox_private_ref` (unique partiel, `WHERE private_ref IS NOT NULL`)	`private_ref`	Une création de commande n'est mise en file qu'une fois
`hubrise_outbox`	`idx_hubrise_outbox_order` (partiel, `WHERE sent_at IS NULL`)	`hubrise_order_id, id`	Envoi dans l'ordre, par commande
`webshop_orders`	`idx_webshop_orders_state`	`fiscal_state, resolution`	Liste des commandes en ligne par état
`order_price_changes`	`idx_order_price_changes_order`	`order_id, entry_id`	Remises d'une commande
`eft_payments`	`idx_eft_payments_order`	`order_id`	Paiements carte d'une commande

Table	Index	Colonnes	Raison
eft_payments	idx_eft_payments_open (partiel, WHERE status IN ('sent','unknown'))	status	Paiements qui attendent encore leur réponse

Index implicites utiles : fiscal_events (id, UNIQUE(event_type, sequence_number)), users (insz), translations (UNIQUE(entity_type, entity_id, field, lang)), webshop_orders (order_id, hubrise_order_id).

Des indexes additionnels peuvent être ajoutés en post-cert si des problèmes de performance apparaissent à plus grande échelle (multi-restaurant). Pour le périmètre mono-terminal (~150 tickets/jour pour un restaurant type), aucun index supplémentaire n'est nécessaire.

---

6. Tables des commandes en ligne

Les tables des commandes en ligne (webshop_orders, hubrise_channels, hubrise_cash_payment_refs, hubrise_outbox, hubrise_events, hubrise_connection, order_price_changes, integration_switches, integration_switch_log, catalog_sync_state, migrations 20 à 25 et colonnes de la migration 21) sont décrites dans SCHEMA_BD_FR.md §5. Elles ne sont écrites que si l'intégration est activée pour l'installation (COMMANDES_EN_LIGNE_FR.md §2), sauf le journal de l'interrupteur, qui reçoit au moins la valeur de départ. webshop_orders.order_id référence orders.id ; ses colonnes p_event_id, n_event_id et refund_event_id référencent fiscal_events.id.

---

7. Versionnement du schéma

Le schéma est versionné via pos-db::migrate::run_migrations. La table _migrations_applied enregistre chaque migration appliquée (version, name, applied_at). Aucune modification de schéma post-cert n'est possible sans une nouvelle demande de certification (Article 109 du manuel).

Version actuelle du schéma : migrations 1 à 11, puis 20 à 28, puis 70 (vingt et une migrations ; 12 à 19 sont réservées aux correctifs du binaire certifié) : initial-schema, catalogue-fiscal-fields, booking-periods, fiscal-archive-append-only, ticket-deliveries, original-prints, german-language, meal-voucher-payments, shifts-and-pre-bill, pending-signs, table-plan, hubrise-transport, catalog-sync, webshop-orders, price-changes, platform-charges, integration-switch, product-codes, eft-payments, presence-registration, staff-badges - liste exacte dans SCHEMA_BD_FR.md §6 et crates/pos-db/src/migrate.rs (MIGRATIONS, HUBRISE_MIGRATIONS, CATALOG_MIGRATIONS). Les migrations 20 à 28 et 70 font partie du binaire certifié.